Unit 2: Python Programming

Complete exercise solutions — MCQs, Short Questions, Long Questions

EXERCISE SOLUTIONS — Unit 2: Python Programming

Multiple Choice Questions

1. An action needed during Python installation to run from the command line easily is:

- a) Uncheck 'Add Python to PATH'
- b) Choose a different IDE
- c) Check 'Add Python to PATH'
- d) Install only the IDE

Answer: c) Check 'Add Python to PATH'

2. A valid variable name in Python is:

- a) variable
- b) 1variable
- c) variable-name
- d) variable name

Answer: a) variable

3. Output of the following code: age = 25; print('Age: ', age)

- a) Age: 25
- b) 25 Age
- c) age
- d) Error

Answer: a) Age: 25

4. The operator used for exponentiation in Python is:

- a) *
- b) **
- c) ^
- d) //

Answer: b) **

5. A loop used to iterate over a collection such as lists is:

- a) while
- b) for
- c) do-while

StudySapiens.com

• d) repeat

Answer: b) for

6. The range() function is used to:

• a) Generates a list of numbers

• b) Creates a sequence of numbers

• c) Calculates the sum of numbers

• d) Prints a range of numbers

Answer: b) Creates a sequence of numbers

7. A keyword used to define a function in Python is:

• a) define

• b) function

• c) def

• d) func

Answer: c) def

8. Output of the code: temperature, humidity, wind_speed = 25, 60, 15; print('Hot and humid!' if temperature > 30 and humidity > 50 else 'Warm and breezy' if temperature == 25 and wind_speed > 10 else 'Cool and dry' if temperature < 20 and humidity < 30 else 'Moderate')

• a) Hot

• b) Warm

• c) Cool

• d) Nothing

Answer: b) Warm

9. The operation used to combine two lists in Python is:

• a) combine()

• b) concat()

• c) +

• d) merged()

Answer: c) +

Short Questions

1. Explain the purpose of using comments in Python code.

Comments are lines that the Python interpreter ignores. Their purposes are: (1) to explain what the code does, making it easier for others (and yourself later) to understand; (2) to document important decisions and logic; (3) to temporarily disable a line of code without deleting it. Single-line comments start with #, and multi-line comments use triple quotes (''' or """). Good comments improve code readability and maintainability.

2. Describe the difference between integer and float data types in Python. Provide an example of each.

An integer (int) stores whole numbers without a decimal part, e.g., age = 25. A float stores numbers with a decimal part, e.g., price = 19.99. Key differences: int values are exact whole numbers, while float values can represent fractions but may have small precision errors. Python converts between them using int() and float(). For example, int(3.9) gives 3, and float(5) gives 5.0.

3. Define operator precedence and give an example of an expression where operator precedence affects the result.

Operator precedence is the order in which Python evaluates operators when an expression contains more than one. The order is: parentheses first, then exponentiation (**), then multiplication/division/modulo (*, /, %), then addition/subtraction (+, -). Example: 2 + 3 * 4 evaluates to 14, not 20, because multiplication happens before addition. If parentheses are used, (2 + 3) * 4 evaluates to 20. This shows how precedence changes the result.

4. How does the short-hand if-else statement differ from the regular if-else statement?

The regular if-else statement uses multiple lines with indentation and can contain several statements. The short-hand (ternary) if-else fits on one line in the form: value_if_true if condition else value_if_false. For example, result = 'Pass' if marks >= 50 else 'Fail'. The short-hand is useful for simple assignments, while the regular form is better for complex logic with multiple statements.

5. Explain the use of the range() function in a for loop.

The range() function generates a sequence of numbers that a for loop can iterate over. It takes up to three arguments: range(start, stop, step). For example, range(5) gives 0, 1, 2, 3, 4; range(1, 6) gives 1 to 5; range(0, 10, 2) gives even numbers 0, 2, 4, 6, 8. It is commonly used when you need to repeat an action a known number of times or iterate over indices, e.g., for i in range(5): print(i).

6. Explain how default parameters work in Python functions.

A default parameter is a parameter with a pre-assigned value in the function definition. If the caller does not provide an argument, the default value is used. Example: def greet(name='Student'): print('Hello, ' + name). Calling greet() outputs 'Hello, Student!', while greet('Umer') outputs 'Hello, Umer!'. Default parameters make functions more flexible and reduce the need for multiple function versions. Note: parameters with defaults must come after parameters without defaults.

7. Explain why modular programming is useful in Python.

Modular programming means dividing a program into separate, reusable functions or modules. It is useful because: (1) it breaks complex problems into smaller, manageable parts; (2) code becomes reusable — a function written once can be used many times; (3) it is easier to debug and maintain, since each module has a single responsibility; (4) teams can work on different modules simultaneously. Python supports this through functions (def) and modules (import).

8. Explain the difference between a class and an object in Python.

A class is a blueprint or template that defines the attributes (data) and methods (functions) of a type of thing. An object is an instance created from a class — a real thing built from the blueprint. Example: class Car defines what a car is; my_car = Car() creates an object. The class is defined once, but many objects can be created from it, each with its own attribute values. The __init__ method is used to initialise object attributes when the object is created.

Long Questions

1. Evaluate the following Python expressions: (a) (18/3 + 4%2) - (2*7 - 3) (b) (25 + 3*4%2 - 6)/(2*3 + 1) - 7 (c) (12 + 6*5 - 2)*2/((4*2 - 7) + 10) (d) 45/2*2 + 3*4 + 8*(7 - 3)

(a) (18/3 + 4%2) - (2*7 - 3) = (6 + 0) - (14 - 3) = 6 - 11 = -5. (b) (25 + 3*4%2 - 6)/(2*3 + 1) - 7: first 3*4%2 = 12%2 = 0, so numerator = 25 + 0 - 6 = 19; denominator = 6 + 1 = 7; 19/7 ≈ 2.714; 2.714 - 7 ≈ -4.286. (c) (12 + 6*5 - 2)*2/((4*2 - 7) + 10): numerator = (12 + 30 - 2)*2 = 40*2 = 80; denominator = (8 - 7) + 10 = 11; 80/11 ≈ 7.27. (d) 45/2*2 + 3*4 + 8*(7 - 3) = 22.5*2 + 12 + 8*4 = 45 + 12 + 32 = 89.

2. Translating the following mathematical expressions to Python syntax: (a) 5x^4 - 2/6 (b) 7 + 2*(something)

Note: The textbook expressions are partially garbled in print. The general method: use ** for powers, * for multiplication, / for division, and parentheses to preserve order. For example, 5x^4 - 2/6 becomes 5 * x**4 - 2/6. Always use parentheses to match the mathematical grouping, and remember that in Python, / always gives a float result.

3. Explain the concept of variables in Python.

A variable is a named container that stores a value in memory. In Python, variables are created simply by assigning a value: age = 25. Python is dynamically typed, meaning you do not declare the type — it is inferred from the value. Variables can hold different data types: int, float, str, bool, list, etc. The same variable can be reassigned to a new value, even of a different type. Variable names must start with a letter or underscore, can contain letters, digits, and underscores, and are case-sensitive. Using descriptive names makes code readable.

4. Write a Python program that takes a number as input and checks whether it is positive, negative, or zero using an if-elif-else statement.

num = float(input('Enter a number: ')) if num > 0: print('The number is positive.') elif num < 0: print('The number is negative.') else: print('The number is zero.') Explanation: The program reads a number using input() and converts it to float. The if-elif-else structure checks each condition in order: if greater than zero, it is positive; elif less than zero, it is negative; otherwise (exactly zero), the else block runs.

5. Write a Python program using a while loop that prints all the odd numbers between 1 and 100. Also, count and print the total number of odd numbers.

count = 0 i = 1 while i <= 100: if i % 2 != 0: print(i) count += 1 i += 1 print('Total odd numbers:', count) Explanation: The while loop runs from 1 to 100. The condition i % 2 != 0 checks if the number is odd (remainder 1 when divided by 2). Each odd number is printed and the counter is incremented. After the loop, the total count (50) is printed.

Studysapiens.com